“Edge-Driven, Cloud-Native Classroom occupancy monitoring System”

a project work report submitted to

THE NATIONAL INSTITUTE OF ENGINEERING, MYSURU

(An Autonomous Institute under VTU, Belagavi)

In partial fulfilment of the requirements for major-project,

seventh semester

Bachelor of Engineering in

Computer Science and Engineering

Submitted by

Ganesha M	**4NI22CS062**
Harshitha S Shankar	**4NI22CS076**
Hemanth Kumar R	**4NI22CS079**

Under the guidance of

Priyanka R V

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

THE NATIONAL INSTITUTE OF ENGINEERING

(An Autonomous Institution Under VTU)

Mananthavadi Road, Mysuru- 570008

2025-2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

THE NATIONAL INSTITUTE OF ENGINEERING

CERTIFICATE

This is to certify that the project work entitled **"VERILOC: Edge-Driven, Cloud-Native Classroom occupancy monitoring System"** is a bonafide work carried out by **Ganesha M(4NI22CS062), Harshitha S Shankar(4NI22CS076) and Hemanth Kumar R (4NI22CS079)** in partial fulfillment of the requirements for the Phase I Major Project work, Seventh Semester, Computer Science and Engineering, The National Institute of Engineering (Autonomous under VTU) during the academic year **2025–2026**.

Signature of Guide

Priyanka R V
Assistant Professor
Dept. of CS&E
NIE, Mysuru

Signature of the HOD

Dr. Anitha R
Professor & Head
Dept. of CS&E
NIE, Mysuru

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people whose ceaseless cooperation made it possible whose constant guidance and encouragement crown all efforts.

First and foremost, we would like to thank our beloved principal **Dr. Rohini Nagapadma** for being the patron and the beacon light for this project.

We would like to express our sincere gratitude to our HOD **Dr. Anitha R,** Department of CSE, NIE for her relentless support and encouragement.

It gives us immense pleasure to thank our guide **Mrs. Priyanka R V,** Assistant Professor, Department of CSE, NIE for her valuable suggestions and guidance during the process of the project and for having permitted us to pursue work on the subject.

Ganesha M **4NI22CS062**

Harshitha S Shankar **4NI22CS076**

Hemanth Kumar R **4NI22CS079**

ABSTRACT

VERILOC is a real-time classroom availability tracker that integrates IoT hardware (ESP32 with R307 fingerprint sensor and 16×2 LCD) with a MERN-stack web dashboard (MongoDB, Express.js, React, Node.js). The system authenticates authorized personnel via fingerprint, enabling secure local updates of room occupancy which are transmitted over Wi-Fi to the REST API. The backend persists room and activity data in MongoDB while the React dashboard provides authenticated administrators with live room status, analytics, exportable reports, and activity logs. VERILOC reduces scheduling conflicts and improves space utilization by providing immediate occupancy visibility, an auditable activity trail, and role-based access control. Implementation highlights include biometric-secured updates, robust server-side validation, token-based authentication (JWT), and responsive dashboard visualizations. In testing, the system demonstrated sub-second recognition-to-update latency for fingerprint-based status changes and reliable performance under moderate concurrent load. Proposed future enhancements include predictive analytics for occupancy forecasting, mobile push notifications, multi-tenant support, and integration with institutional timetabling systems to further automate room allocation and improve operational efficiency.

TABLE OF CONTENTS

LIST OF FIGURES

Chapter 1

INTRODUCTION

1.1 Overview

In today's academic landscape, the efficient use of institutional resources such as classrooms has become increasingly important. Universities and colleges often face challenges when it comes to managing classroom availability, especially during unplanned lectures, guest sessions, or rescheduled events. Many times, students and faculty members waste valuable time looking for vacant classrooms or clarifying the status of a room through word of mouth. Traditional systems, which rely on static timetables, manual updates on notice boards, or direct inquiries, fail to provide real-time information. This leads to confusion, underutilization of infrastructure, and disruptions in academic activities. To address these shortcomings, there is a growing need for a Classroom Availability Tracker that integrates modern technologies and provides an automated, reliable, and transparent solution. The proposed project makes use of an ESP32 microcontroller integrated with a fingerprint sensor and an LCD display, combined with a MERN-based web application that allows real-time status updates. By fusing embedded systems with cloud technologies, the project ensures that classroom availability is not only monitored accurately but also made accessible to everyone through a centralized digital platform.

1.2 Objective

The primary objective of this project is to design and develop a robust system that simplifies and digitizes the management of classroom availability within educational institutions. The system aims to provide faculty-exclusive control by enabling only authorized personnel to update the occupancy status of classrooms through a fingerprint authentication mechanism. Once authenticated, faculty members can easily toggle the room status using a simple push-button interface, ensuring quick and hassle-free operation. An LCD display provides immediate visual confirmation of the room's availability within the classroom itself. At the same time, the update is securely transmitted to a cloud database, which forms the foundation of a web application built using the MERN stack. This ensures that students, administrators, and other faculty can access the most recent status information from any device with an internet connection. The objectives of the project also include minimizing manual intervention, enhancing transparency in room allocation, and building a scalable solution that can be deployed across multiple classrooms with minimal cost and effort.

1.3 Need for the Project

The need for this project arises from the fact that current methods of classroom management are inefficient and incapable of handling the dynamic nature of academic schedules. Institutions often depend on manual systems such as paper-based notices, static timetables, or ad-hoc communication between faculty, which are prone to errors and delays. For example, when a class is unexpectedly canceled, the timetable does not reflect this change, leaving the room marked as "occupied" even though it is free. This results in wasted opportunities where available classrooms go unused while students and faculty continue to struggle to find space. Additionally, the absence of an authentication mechanism in the existing process means that anyone can update or miscommunicate room status, which compromises accuracy and trust. In contrast, modern institutions are shifting toward digital solutions to streamline operations and improve efficiency. A Classroom Availability Tracker not only addresses the gaps in the existing system but also represents a step forward in digital transformation for academia, where real-time monitoring, reliability, and user accessibility become the norm.

1.4 Existing System and Drawbacks

The existing systems employed in many educational institutions to manage classroom availability are predominantly manual and suffer from significant limitations due to the lack of automation. In most colleges, faculty members or administrative staff are responsible for recording room occupancy status on traditional mediums such as whiteboards, pinboards, or paper charts placed outside classrooms. This approach is inherently unreliable, as the information can only be verified by physically visiting the classroom, which consumes valuable time and effort. Faculty and students alike often have to move between multiple classrooms to confirm availability, causing delays and inefficiencies in daily academic activities.

Moreover, manual systems fail to capture real-time changes in classroom usage. Situations such as rescheduled classes, canceled lectures, or extended sessions are not promptly reflected, making the recorded data obsolete and often misleading. This lack of immediacy can lead to confusion, miscommunication, and underutilization of resources. Another critical shortcoming is the absence of authentication or controlled access. Since anyone can modify the room status on paper charts or boards, the system lacks accountability, and there is no assurance that the information is accurate or trustworthy.

Static timetables provided at the start of the semester further exacerbate the problem, as they rarely reflect dynamic changes in classroom allocation, leaving students and faculty uncertain about actual availability. Additionally, the absence of a centralized, digital platform accessible remotely prevents students and staff from checking classroom status conveniently from outside the campus. This limitation not only reduces operational efficiency but also hampers planning and coordination in an increasingly dynamic academic environment. In essence, the existing manual systems are outdated, inefficient, and incapable of providing accurate, real-time, and secure information about classroom occupancy, highlighting the urgent need for an automated, centralized solution.

1.5 Proposed System

The VERILOC – Classroom Occupancy Monitoring System is an advanced, integrated solution designed to address the inefficiencies of traditional and semi-automated classroom management methods. By combining IoT hardware with a cloud-based MERN stack (MongoDB, Express.js, React, Node.js) software platform, the system delivers a secure, real-time, and scalable approach to monitoring and managing classroom occupancy. It represents a key step toward building smart, technology-driven infrastructures in modern educational institutions.

At the hardware level, the system employs the ESP32 microcontroller, a powerful and cost-effective device featuring built-in Wi-Fi and seamless integration with external sensors. For secure faculty authentication, an R307 fingerprint sensor is interfaced with the ESP32, ensuring that only authorized users can modify classroom status. This biometric verification mechanism enhances data integrity and prevents unauthorized access or tampering.

Upon successful authentication, the faculty member interacts with push buttons to toggle the classroom status between “Vacant” and “Occupied.” The selected status is instantly displayed on an LCD screen, offering immediate visual confirmation to all occupants. Simultaneously, the ESP32 transmits the updated data to the central server via Wi-Fi, ensuring real-time synchronization across all connected systems. This automated data flow minimizes human error and ensures that occupancy details remain current and accurate at all times.

The backend system, developed using Node.js and Express.js, processes incoming data from IoT devices and securely communicates with the MongoDB database, which stores all information related to administrators, classrooms, and activity logs

The backend also manages authentication through JWT (JSON Web Tokens) and ensures secure API communication between the hardware and web application.

On the frontend, a responsive React.js dashboard allows administrators, faculty, and students to view classroom availability, manage rooms, and analyze occupancy statistics. The user interface is designed with Tailwind CSS for a clean and modern experience, supporting features such as dark mode, graphical analytics, and real-time data visualization.

Security and reliability are central to the system's design. Fingerprint-based authentication prevents unauthorized updates, while all transactions are securely logged in the database, enabling traceability and accountability. By leveraging open-source tools and affordable IoT components, VERILOC achieves an ideal balance between cost-effectiveness, scalability, and performance, making it suitable for institutions of all sizes.

Another key strength of the proposed system is its real-time data synchronization between the ESP32 devices, backend server, and frontend dashboard. Any change in room status is reflected instantly across the platform, improving communication efficiency and minimizing confusion or scheduling conflicts. This transparency promotes effective resource utilization and enhances operational efficiency across departments.

Additionally, the system's modular and extensible architecture supports future enhancements like:

- AI-based occupancy prediction for smarter scheduling,
- Integration with institutional timetables, and
- Advanced analytics dashboards for administrative insights.

These features position VERILOC as a forward-looking solution adaptable to future campus automation needs. the proposed VERILOC system offers a secure, intelligent, and efficient method for real-time classroom occupancy tracking. It successfully integrates biometric authentication, IoT connectivity, and cloud-based analytics to deliver a smart campus ecosystem. The system not only optimizes resource utilization but also enhances transparency, reliability, and overall productivity within educational institutions.

Chapter 2

LITERATURE SURVEY

[1] "Smart Classroom Occupancy Detection Using IoT Sensors":

Real-world implementations—such as low-cost Bluetooth Low Energy (BLE) occupancy detection—offer highly accurate, cost-effective room occupancy estimation. These systems can achieve nearly 97.97% accuracy in detecting presence and estimate the number of occupants with an average error of only 0.32 persons, making them practical for real-world deployment. Another camera-based approach using Raspberry Pi and YOLOv3 models has achieved over 90% accuracy in detecting occupancy across diverse classroom environments. While these solutions excel at sensing occupancy, they lack user authentication and secure status updates—crucial for your system's requirement of verified toggling of room availability.

[2] "Design and Implementation of a Biometric Authentication System for Classroom Access:

Biometric authentication, particularly fingerprint-based systems, has gained significant traction in educational settings. Solutions integrating Arduino or NodeMCU microcontrollers with fingerprint sensors are commonly used to log student attendance and transmit records to cloud servers or institutional databases for easy management and analytics. Surveys and practical deployments have highlighted the efficiency, accuracy, and user convenience of biometric authentication in identity verification and access control, supported by research from platforms like SpringerLink and MDPI. However, most existing systems remain narrowly focused on attendance tracking and access logging, without integration into occupancy detection systems or real-time availability management. This limits their potential for dynamic classroom monitoring and resource scheduling in more advanced automated infrastructures.

[3] "An IoT Framework for Real-Time Resource Scheduling in Educational Institutions":

IoT frameworks for educational resource scheduling typically leverage a combination of environmental sensors (such as CO_2, temperature, and humidity detectors) or network-based data collection methods like Wi-Fi sniffing. These data streams are often paired with cloud dashboards that provide live monitoring and predictive models, such as K-NN or SVM algorithms, achieving moderate accuracy rates of approximately 88%. Such systems enable real-time visibility into classroom utilization, helping administrators make informed decisions. Despite their strengths, these frameworks often lack authentication mechanisms and secure control protocols, meaning updates to

room availability cannot be restricted to authorized personnel. This gap significantly limits their application in high-integrity systems where security and accountability are critical.

[4] "Low-Cost Embedded Systems for Smart Campus Infrastructure":

At the hardware level, low-cost embedded systems play a pivotal role in the design of smart campus solutions. Sensors such as PIR (Passive Infrared), ultrasonic, microwave, and environmental modules are widely deployed for detecting motion, occupancy, and other environmental parameters in classrooms and campus spaces. Coupled with edge processing capabilities on microcontrollers like the ESP32, these systems enable real-time, low-latency occupancy detection while keeping costs manageable. However, these solutions often operate in isolation, focusing solely on detection. They typically lack integration with authentication protocols or secure backend synchronization, making them unsuitable for contexts where verified user interaction and centralized management are essential for reliability and security.

[5] "Secure IoT Authentication Using Biometric Sensors":

In IoT environments, biometric authentication paired with robust cryptographic protocols provides a strong foundation for security and reliability. Advanced implementations leverage AES-256 encryption for secure storage and transmission of biometric templates, reducing the risk of tampering, spoofing, or unauthorized access. Research in this area highlights the ability of such systems to deliver high accuracy and robust protection against cyber threats, ensuring that only authorized users can perform sensitive operations. They do not integrate seamlessly with real-time occupancy detection systems or automated scheduling workflows, leaving a gap for more comprehensive, interconnected solutions in academic environments.

[6] "Real-Time Attendance and Availability Systems in Smart Environments":

Recent studies highlight significant progress in developing integrated smart classroom systems that combine environmental sensing, microcontrollers, and cloud platforms for seamless real-time monitoring. For example, systems using Micro Python-enabled controllers like the Raspberry Pi Pico W with cloud services such as ThingSpeak provide live data visualization, while biometric fingerprint authentication ensures secure and verified user interactions. These prototypes demonstrate how IoT and cloud-based architectures can streamline attendance tracking and enhance data accessibility and transparency for both staff and students. They often lack integration with centralized scheduling platforms or robust authorization protocols, which are essential for scaling to institution-wide deployments and ensuring data security and reliability.

Chapter 3

SYSTEM REQUIREMENTS AND SPECIFICATIONS

3.1 Software Requirements

- **Programming Environment:** Arduino IDE (for ESP32 firmware)
- **Backend Framework:** Node.js with Express.js (API handling)
- **Frontend Framework:** React.js (web application dashboard)
- **Database:** MongoDB (cloud-hosted, for storing classroom status, user data, and logs)
- **Authentication:** JWT (JSON Web Token) for securing API endpoints
- **Testing Tools:** Postman for backend API testingg.
- **Version Control:** Git for collaborative development
- **Networking:** Stable Wi-Fi connection for ESP32 to communicate with the backend

3.2 Hardware Requirements

- **Microcontroller:** ESP32 (core processing unit for fingerprint authentication)
- **Biometric Sensor:** R307s Fingerprint Sensor for secure faculty authentication
- **Display:** 16*2 LCD display for real-time status feedback
- **Input Device:** Push buttons for toggling classroom status (Vacant/Occupied)
- **Power Supply:** 5V USB adapter or portable power bank
- **Supporting Components:** Jumper wires

3.3 Functional Requirements

- **Faculty Authentication:** The system should authenticate faculty using a fingerprint sensor to ensure only registered staff can update the classroom status.
- **Classroom Status Update:** After authentication, faculty can toggle the status between Vacant and Occupied using a push button for quick updates
- **Local Display Feedback:** An LCD display must show the current status in real time, giving instant visual confirmation inside or near the classroom.
- **Cloud Synchronization:** The ESP32 should send instant status updates to the backend API, where data is stored in MongoDB for reliability and centralized access.
- **Web Application**: A React-based dashboard should allow students and staff to view live classroom availability from any device.

- **Audit Logging:** Every status change should be timestamped and logged for monitoring and administrative reviews.
- **Scalability**: The system should handle multiple classrooms and devices efficiently, allowing easy expansion without performance issues.

3.4 Non-Functional Requirements

- **Security:** Ensure fingerprint authentication for faculty and use JWT-secured APIs to prevent any unauthorized access or tampering with classroom data.
- **Reliability:** The system must stay operational during network disruptions, queuing updates locally and syncing them once connectivity is restored.
- **Performance:** Status changes should update on the web dashboard within 2–3 seconds to maintain real-time accuracy.
- **Usability:** Provide a minimal interface for faculty (just the fingerprint sensor and toggle button) and a simple, intuitive dashboard for students and staff to check availability easily.
- **Scalability:** The system should support multiple classrooms with a centralized backend, allowing smooth expansion without performance degradation.
- **Compatibility** The web application must function seamlessly on modern browsers and mobile devices, ensuring universal accessibility.
- **Maintainability:** Code should be modular, well-structured, and documented, making future updates and feature enhancements straightforward.

Chapter 4

SYSTEM DESIGN AND ANALYSIS

4.1 High Level Diagram

At the architectural level, the system consists of multiple interconnected layers. The input layer involves the faculty interacting with the fingerprint sensor and buttons, where their identity is authenticated and the status is toggled. The processing layer within the ESP32 validates inputs, controls the LCD display, and prepares the data for communication. The communication layer ensures secure transmission of the room status over Wi-Fi to the backend server. The backend layer, built with Node.js and Express, receives the update, validates it, and forwards it to the database layer, which is a MongoDB instance hosted in the cloud.

Finally, the frontend layer is developed using React, providing an accessible interface where students, staff, and administrators can view classroom availability in real time. This layered approach ensures modularity, scalability, and efficiency.

At the high level, the system is organized into multiple layers, each handling a specific function to ensure smooth integration between hardware and software components.

- Input Layer – This is the first point of interaction for faculty members. They scan their fingerprint using the R307s sensor, and upon successful authentication, use the push buttons to toggle the classroom status as either Vacant or Occupied.
- Processing Layer – The ESP32 microcontroller validates the fingerprint data, interprets the button input, updates the LCD display to provide immediate feedback, and prepares the status message for transmission.
- Communication Layer – This layer handles the Wi-Fi communication between the ESP32 and the backend server. It ensures that the status update is transmitted securely and reliably to the cloud.
- Backend Layer – Implemented with Node.js and Express.js, this layer processes the incoming data, validates the request, and pushes the update to the cloud-hosted MongoDB database.
- Database Layer – MongoDB stores the current status along with timestamps and logs, enabling both real-time updates and historical tracking for administrative purposes.
- Frontend Layer – Built with React, the web interface displays the latest classroom status to students, faculty, and administrators. It provides a user-friendly, responsive dashboard that can be accessed from desktops or mobile devices.

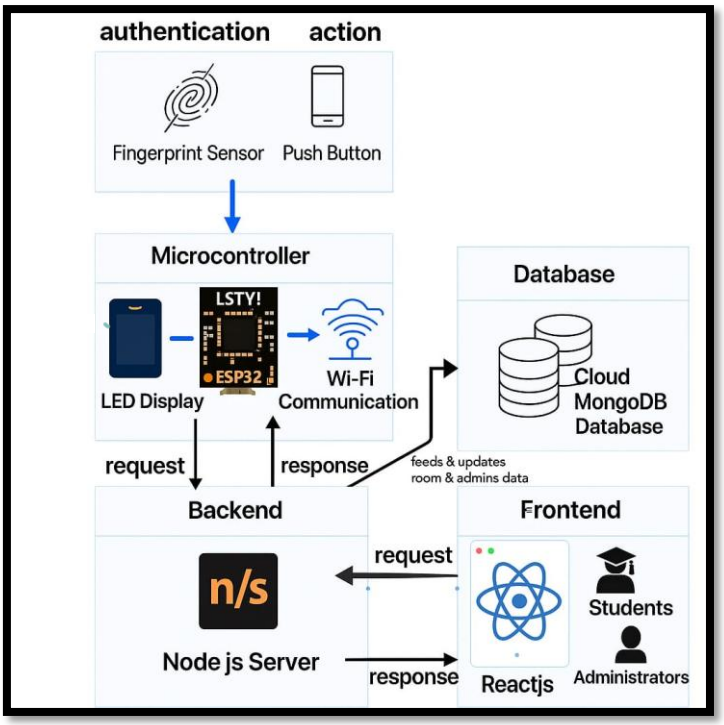


Fig 4.1 – System Interaction Sequence

4.2 Detailed Level Diagram

At the detailed level, the system can be described using diagrams such as use case and sequence diagrams. In the use case diagram, the main actors are the faculty, students, and system administrators. Faculty members authenticate themselves, update classroom status, and confirm it via the LCD. Students and administrators primarily interact with the web app to check availability. The sequence diagram depicts the step-by-step process where a faculty member authenticates using the fingerprint sensor, the ESP32 verifies the input, allows status toggling, updates the LCD display, and simultaneously sends the update to the backend. The backend processes this request, stores it in MongoDB, and makes it accessible through the web frontend. This flow ensures end-to-end consistency across all modules.

The detailed-level design breaks down the internal workflows and interactions between components, ensuring end-to-end consistency and reliability.

- Authentication and Control – The process starts when a faculty member places their finger on the R307s fingerprint sensor. The ESP32 verifies the fingerprint data against the stored template.
- Status Toggle and Display – Upon authentication, the faculty member presses the button to switch the status. The ESP32 interprets this input and updates the LCD display inside the classroom, providing immediate visual confirmation of the room's current state.

- Data Transmission - the ESP32 sends the updated status via Wi-Fi to the backend API.
- Backend and Database Synchronization – The backend receives the request, validates the user action, and updates the MongoDB database with the new status and a timestamp. This ensures accurate, real-time logs of all interactions.
- Frontend Interaction – The web dashboard fetches the updated data instantly from the database and reflects the change to students and administrators. This provides a synchronized and transparent view of occupancy information.
- System Workflow – The entire sequence, from fingerprint authentication to status display and database update, is streamlined to execute within a few seconds, ensuring that the system maintains real-time performance and reliability.

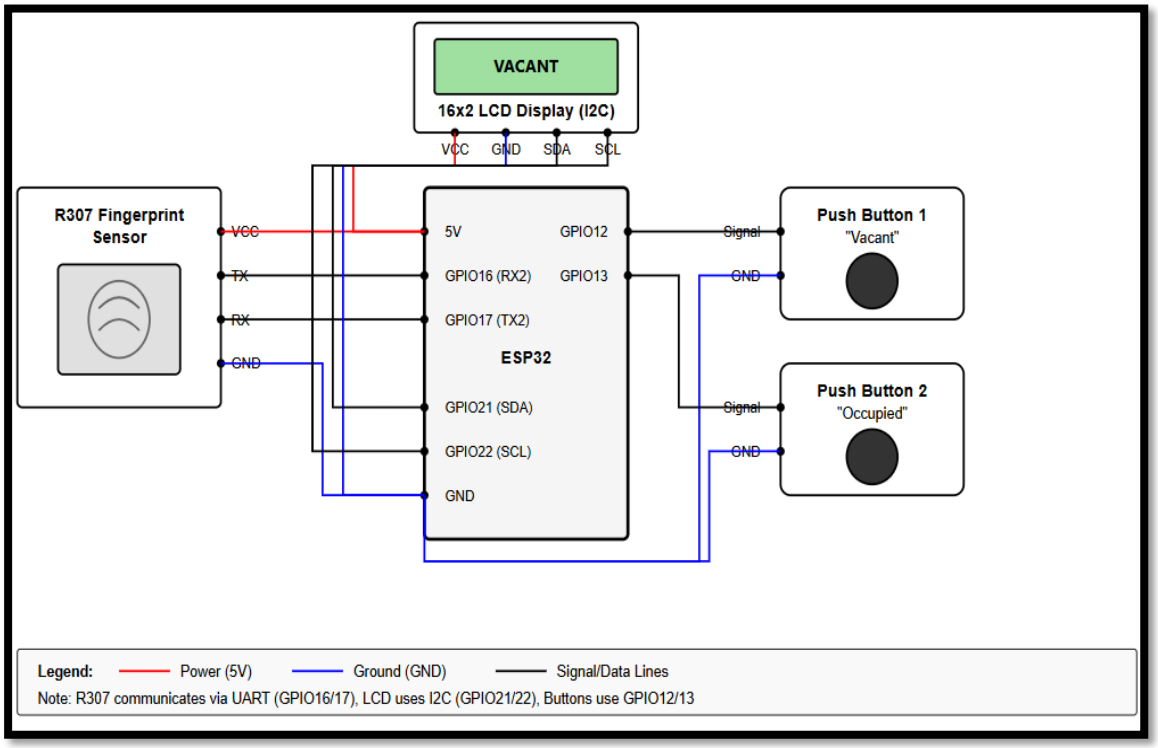


Fig 4.2 – ESP32 Component Connection Diagram

The circuit diagram illustrates the complete hardware architecture of the toilet occupancy monitoring system. The ESP32 microcontroller serves as the central processing unit, interfacing with four key components: an R307 fingerprint sensor for user authentication via UART communication (GPIO16/17), a 16x2 LCD display with I2C interface (GPIO21/22) for status visualization, and two push buttons connected to GPIO12 and GPIO13 for manual occupancy status updates ("Vacant" and "Occupied"). All components share common power (5V) and ground connections from the ESP32, which is powered via USB. The fingerprint sensor enables secure access control, while the display provides real-time feedback to users, and the push buttons offer a mechanism for system operation.

4.3 Data Flow Diagram

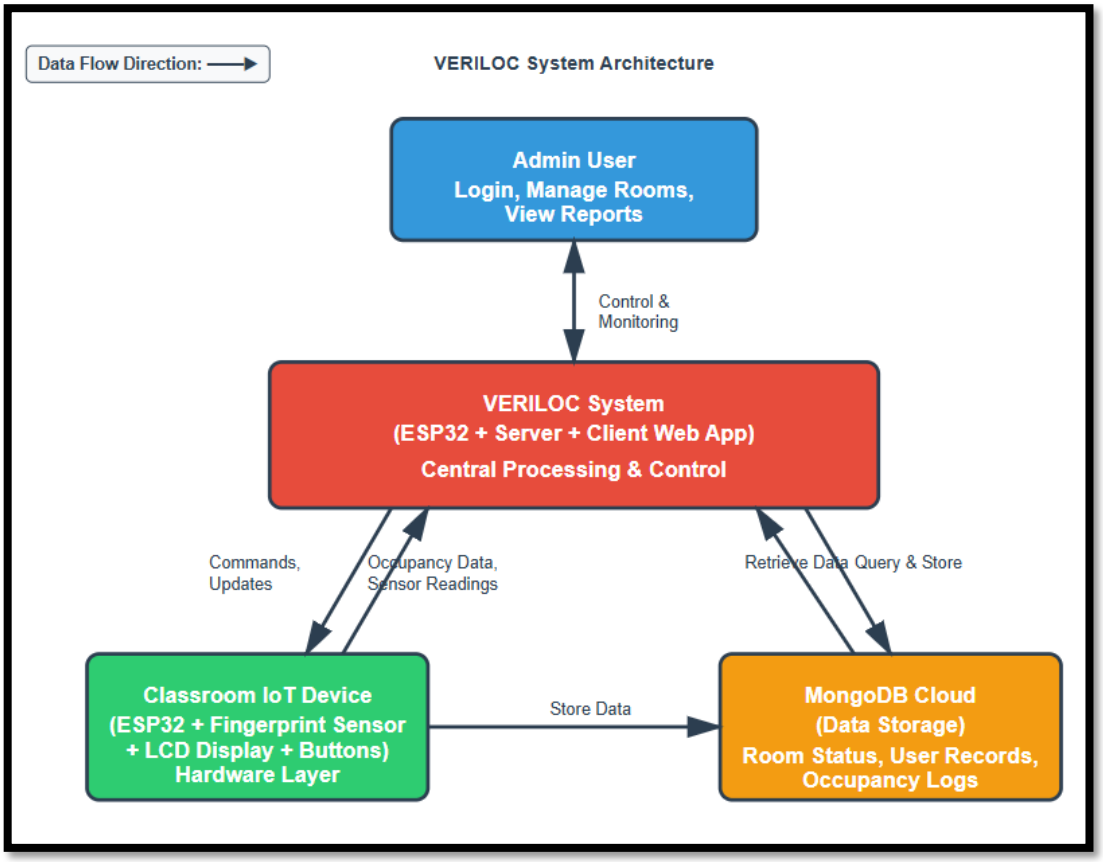


Fig 4.3.1 – Level 0 Data Flow Diagram

The fig ----describes the level 0 data flow diagram illustrating External Entities & Data Flow. Admin/User interacts with the web client (React app) to view and manage occupancy. ESP32 Hardware sends real-time occupancy data (fingerprint or room status) to the backend server via REST API/Wi-Fi. Server (Node.js + Express) processes data, authenticates users, and stores records in MongoDB. The Client (React) fetches and visualizes this data for real-time monitoring.

The fig---- describes the level 1 data flow diagram and it illustrates the overall working of the Veriloc system. The Admin/User (Web UI) interacts with the system through the web dashboard to log in, view room occupancy, manage rooms, and generate reports. These actions send REST API requests to the Express.js Server, which acts as the system's controller. The Express.js Server processes these requests using various controllers (Authentication, Room, and Activity) and communicates with the MongoDB Database to store and retrieve data related to admins, rooms, and occupancy logs. The ESP32 Hardware collects real-time occupancy data through sensors and sends it to the server via HTTP POST (JSON) requests. It also receives commands or configurations from the server.

Finally, the Client Web Dashboard, built with React.js, fetches data from the server to display live occupancy information, graphs, and analytics to the users, ensuring real-time monitoring and control.

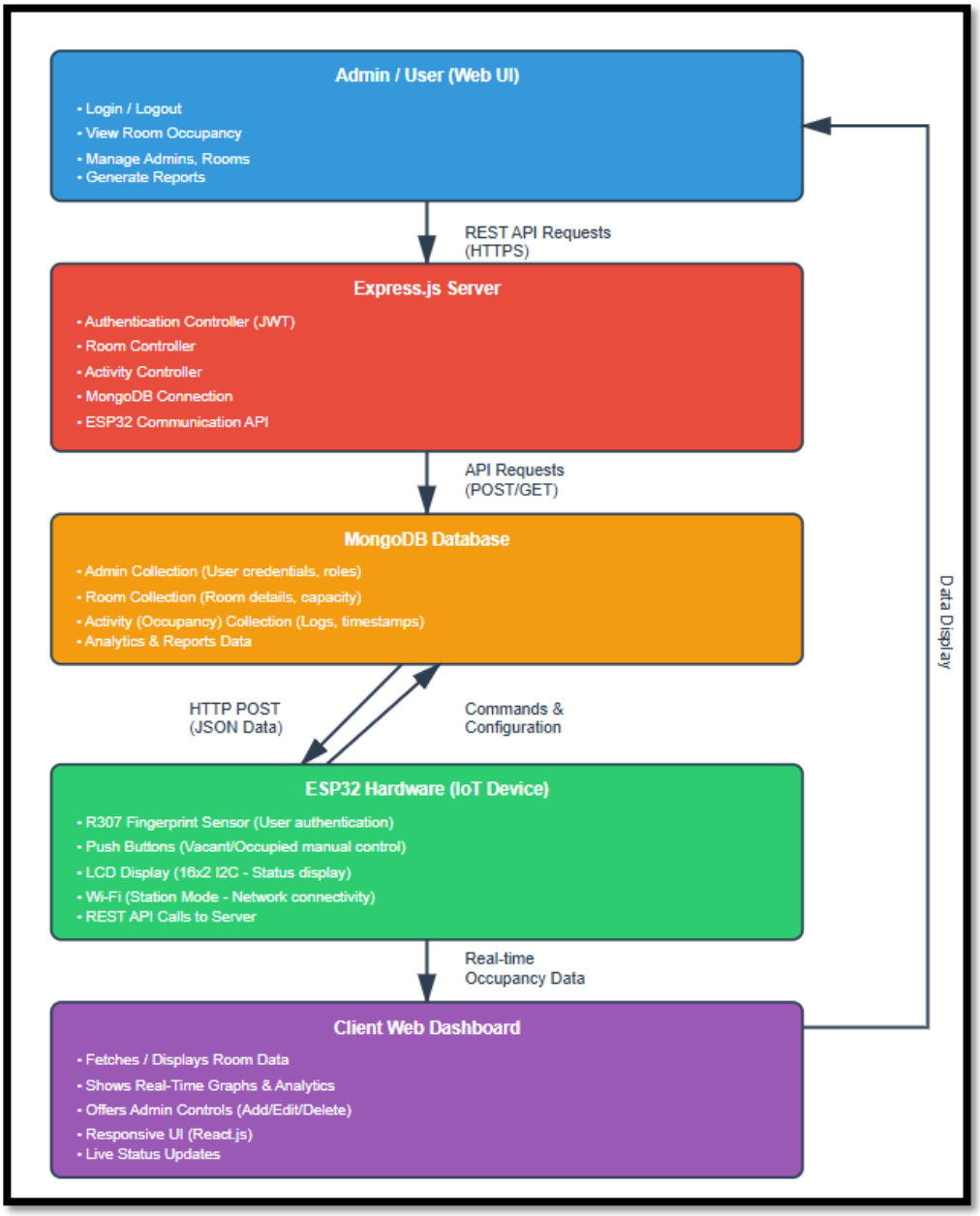


Fig 4.3.2 – Level 1 Data Flow Diagram

4.4 Use Case Diagram

The Use Case Diagram of the VERILOC Classroom Availability Tracker illustrates the interaction between the system and its external actors — Admin, Student, ESP32 Device, and Server. It highlights how hardware, software, and users collaborate to enable efficient and automated classroom monitoring.

The system primarily involves four key actors: Admin, Student, ESP32 Device, and Server. Each actor has a specific set of use cases that describe their interaction with the system. The admin is the main user responsible for managing classroom data, updating occupancy status, and generating reports. Through a secure login, the admin can access features such as managing classroom details, updating room availability after successful fingerprint verification, and viewing analytical reports on classroom usage trends.

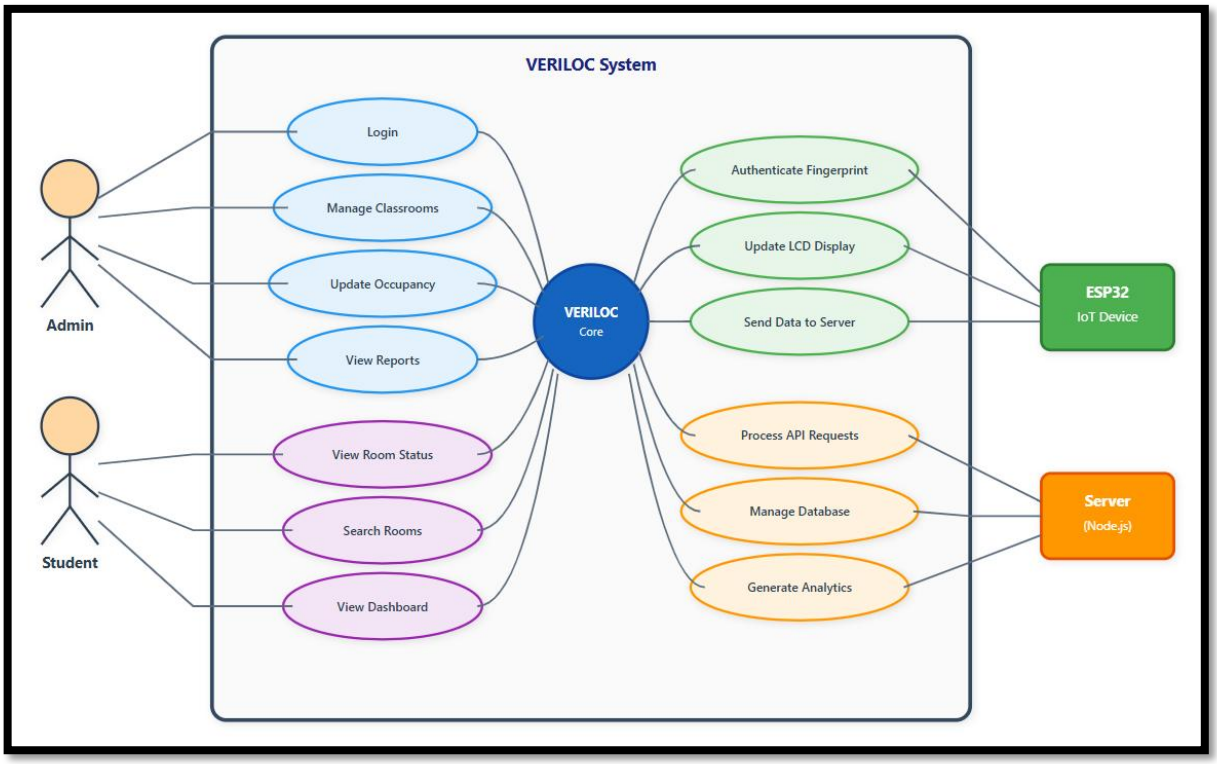


Fig 4.4 – Use Case Diagram

The student interacts with the system to check room availability in real time, search for vacant classrooms, and access the occupancy dashboard. This feature enhances convenience and time management within the campus.

The ESP32 IoT Device acts as the hardware interface, performing fingerprint authentication, updating the LCD display, and transmitting occupancy data to the server via Wi-Fi, ensuring automated and accurate data collection.

The Server handles backend operations such as processing API requests, managing the MongoDB database, and providing analytical data to the admin dashboard.

At the center, the VERILOC Core System integrates all components to maintain real-time synchronization between hardware and software, ensuring a secure, automated, and user-friendly classroom monitoring experience.

Chapter 5

IMPLEMENTATION

5.1 Backend

MongoDB connection and server bootstrap.

This snippet shows how the VERILOC server connects to MongoDB using Mongoose and initializes the Express app. It loads environment variables, sets up a health-check route, and connects securely to the database. After a successful connection, it automatically creates a default admin user if none exists. Finally, the server starts on a defined port, ensuring both the backend and database are ready for use.

```
const express = require("express");
const mongoose = require("mongoose");
const dotenv = require("dotenv");
const { createSuperAdmin } = require('./utils/autoSetup');

dotenv.config({ path: './.env' });

const app = express();
app.use(express.json());

// Health-check
app.get("/api/health", (req, res) => {
  res.json({ message: "VERILOC API is running!", timestamp: new Date() });
});

// MongoDB connection
const MONGODB_URI = process.env.MONGODB_URI || "mongodb://127.0.0.1:27017/veriloc";
mongoose.connect(MONGODB_URI, { useNewUrlParser: true, useUnifiedTopology: true })
  .then(async () => {
    console.log("Connected to MongoDB");
    // Auto-create super admin if not exists
    await createSuperAdmin();
  })
  .catch((error) => {
    console.error("MongoDB connection error:", error);
    process.exit(1);
  });

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`VERILOC server running on port ${PORT}`);
});
```

Fig 5.1.1 – MongoDB connection and server bootstrap.

JWT authentication middleware

This middleware handles JWT (JSON Web Token) authentication for the VERILOC system. It validates tokens sent in the request header as *Bearer tokens* to ensure that only authorized users can access protected routes. When a valid token is detected, the middleware decodes it and attaches the authenticated admin's profile information to the request object, enabling secure user-specific operations. It also includes helper functions for generating new tokens and verifying super-admin privileges, which are essential for managing system access and administrative control. This mechanism ensures a secure, token-based authentication flow across the entire web dashboard and API endpoints.

```
// middleware/auth.js (core methods)
const jwt = require("jsonwebtoken");
const Admin = require("../models/Admin");
const JWT_SECRET = process.env.JWT_SECRET || "fallback_secret_key";

const authenticateToken = async (req, res, next) => {
  try {
    const authHeader = req.headers["authorization"];
    const token = authHeader && authHeader.split(" ")[1]; // "Bearer <token>"
    if (!token) return res.status(401).json({ message: "Access token required" });

    const decoded = jwt.verify(token, JWT_SECRET);
    const admin = await Admin.findById(decoded.adminId).select("-password");
    if (!admin) return res.status(401).json({ message: "Invalid token" });

    req.admin = admin;
    next();
  } catch (error) {
    if (error.name === "TokenExpiredError") return res.status(401).json({ message: "Token expired" });
    return res.status(500).json({ message: "Token verification failed" });
  }
};

const requireSuperAdmin = (req, res, next) => {
  if (!req.admin.isSuperAdmin) return res.status(403).json({ message: "Super admin access required" });
  next();
};

const generateToken = (adminId) => jwt.sign({ adminId }, JWT_SECRET, { expiresIn: "24h" });

module.exports = { authenticateToken, requireSuperAdmin, generateToken };
```

Fig 5.1.2 – JWT authentication middleware

IoT update endpoint - secure room status update from hardware.

Hardware devices (ESP32) post room status changes to this endpoint. The server validates the payload, maps the posted fingerprint ID to an admin, updates the room document, timestamps the change, and logs the activity. This endpoint enforces that only registered fingerprint IDs (admins) can update room state.

```
// routes/rooms.js (relevant extract: /api/rooms/update)
const express = require("express");
const { body, validationResult } = require("express-validator");
const Room = require("../models/Room");
const Admin = require("../models/Admin");
const ActivityService = require("../services/activityService");

router.post(
  "/update",
  [
    body("roomNumber").notEmpty().withMessage("Room number is required"),
    body("status").isIn(["Vacant", "Occupied"]).withMessage("Status must be Vacant or Occupied"),
    body("fingerprintID").isInt({ min: 1000, max: 9999 }).withMessage("Valid fingerprint ID is required"),
  ],
  async (req, res) => {
    try {
      const errors = validationResult(req);
      if (!errors.isEmpty()) return res.status(400).json({ message: "Validation failed", errors: errors.array() });

      const { roomNumber, status, fingerprintID } = req.body;

      // Verify fingerprint -> admin
      const admin = await Admin.findOne({ fingerprintID });
      if (!admin) return res.status(403).json({ message: "Unauthorized fingerprint ID" });

      // Find and update room
      const room = await Room.findOne({ roomNumber });
      if (!room) return res.status(404).json({ message: "Room not found" });

      const oldStatus = room.status;
      room.status = status;
      room.timestamp = new Date();
      await room.save();

      // Log activity
      await ActivityService.logRoomStatusChanged(admin._id, roomNumber, oldStatus, status, admin.username);

      res.json({ message: "Room status updated", room });
    } catch (error) {
      console.error("Hardware update error:", error);
      res.status(500).json({ message: "Room update failed" });
    }
  }
);
```

Fig 5.1.3 – IoT update endpoint

Activity logging service

The Activity Logging Service is responsible for maintaining a detailed record of all important system events. It provides a centralized mechanism for back-end controllers to log activities such as admin logins, classroom status updates, data modifications, and system-generated notifications. Each event is stored in a structured format within the database, including details like user ID, timestamp, and action type. This enables efficient tracking of user interactions, enhances system transparency, and supports future analytics or audit reports. Overall, it helps ensure accountability and provides valuable insights into system usage patterns.

```
// services/activityService.js (core functions)
const Activity = require("../models/Activity");

class ActivityService {
  static async logActivity(type, message, adminId = null, roomId = null, metadata = {}) {
    try {
      const activity = new Activity({ type, message, adminId, roomId, metadata });
      await activity.save();
      return activity;
    } catch (error) {
      console.error("Error logging activity:", error);
      // Non-fatal: do not throw to avoid breaking main flow
    }
  }

  static async logRoomStatusChanged(adminId, roomNumber, oldStatus, newStatus, adminUsername) {
    return this.logActivity(
      "room_status_changed",
      `Room ${roomNumber} status changed from ${oldStatus} to ${newStatus} by ${adminUsername}`,
      adminId,
      null,
      { roomNumber, oldStatus, newStatus }
    );
  }

  static async getRecentActivities(limit = 10) {
    return Activity.find({}).populate("adminId", "username").populate("roomId", "roomNumber")
      .sort({ createdAt: -1 }).limit(limit);
  }
}

module.exports = ActivityService;
```

Fig 5.1.4 – Activity logging service

5.2 Frontend

HTTP client on the web dashboard with token handling

This snippet demonstrates how the frontend application manages secure and consistent API communication using an Axios instance. The instance is configured with a base URL pointing to the deployed backend API and sets the appropriate content-type headers for all outgoing requests. It also includes interceptors that automatically attach the JWT retrieved from the browser's local storage to every authenticated request, ensuring secure access to protected routes. In addition, the code handles common client-side errors in a centralized manner. If a request returns a 401 Unauthorized response, the interceptor automatically clears the stored token and redirects the user to the login page. It also captures network-related errors and displays a standard "Network error" message, improving user experience and reliability. This centralized API service allows all components of the React frontend to communicate with the backend while maintaining consistent request handling.

```
// client/src/services/api.js
import axios from "axios";

const API_BASE_URL = "https://veriloc-api.onrender.com/api";

const api = axios.create({
  baseURL: API_BASE_URL,
  headers: { "Content-Type": "application/json" },
});

// Add auth token to each request if present
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Centralized error handling (401 → logout)
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem("token");
      window.location.href = "/login";
    }
    if (!error.response) {
      error.response = { data: { message: "Network error. Please check your connection." } };
    }
    return Promise.reject(error);
  }
);

export default api;
```

Fig 5.2.1 – HTTP client with token authentication

Authentication context (login, token persistence, session check)

The Authentication Context manages user authentication and session handling across the VERILOC application using React's Context API. It maintains the admin's login state, authentication token, and loading status. When the application loads, it automatically validates any existing token stored in the browser's local Storage and retrieves the admin profile from the server. The login function accepts user credentials, communicates with the backend API at /auth/login, and stores the JWT token for subsequent requests. The logout function clears all session data and removes the token from storage. By using the useAuth hook, any component in the application can access authentication state and functions.

```jsx
// client/src/contexts/AuthContext.jsx (core)
import React, { createContext, useState, useEffect, useCallback, useMemo } from "react";
import api from "../services/api";

const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [admin, setAdmin] = useState(null);
  const [loading, setLoading] = useState(true);
  const [token, setToken] = useState(localStorage.getItem("token"));

  // On load, validate token and get admin profile
  useEffect(() => {
    const initAuth = async () => {
      if (token) {
        try {
          const response = await api.get("/auth/me");
          setAdmin(response.data.admin);
        } catch (err) {
          localStorage.removeItem("token");
          setToken(null);
        }
      }
      setLoading(false);
    };
    initAuth();
  }, [token]);

  const login = useCallback(async (username, password) => {
    const response = await api.post("/auth/login", { username, password });
    localStorage.setItem("token", response.data.token);
    setToken(response.data.token);
    setAdmin(response.data.admin);
  }, []);

  const logout = useCallback(() => {
    localStorage.removeItem("token");
    setToken(null);
    setAdmin(null);
  }, []);

  const value = useMemo(() => ({ admin, loading, isAuthenticated: !!admin, login, logout }), [admin, loading, login,

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
};
export const useAuth = () => React.useContext(AuthContext);
```

Fig 5.2.2 – Authentication context

Occupancy Visualization

The dashboard displays a bar chart comparing vacant and occupied rooms using the react-chartjs-2 library. The chart dynamically visualizes aggregated data with color-coded bars (green for vacant, red for occupied) for easy interpretation. Users can also download the chart as a PNG image for reporting, enabling quick sharing and record-keeping of occupancy trends. The visualization helps administrators monitor room utilization in real time and identify patterns in classroom usage. It provides a clear overview of daily occupancy distribution, aiding in effective space management. This feature enhances the dashboard's complete interactivity and supports data Driven decision making. Overall, it offers a simple yet powerful way to analyse and present occupancy data visually.

.

```jsx
// client/src/components/OccupancyGraph.jsx (core)
import { Bar } from "react-chartjs-2";

const OccupancyGraph = ({ data = [] }) => {
  const chartData = {
    labels: data.map(item => item._id),
    datasets: [
      { label: "Vacant Rooms", data: data.map(i => i.vacant), backgroundColor: "rgba(34,197,94,0.8)" },
      { label: "Occupied Rooms", data: data.map(i => i.occupied), backgroundColor: "rgba(239,68,68,0.8)" }
    ]
  };

  // button handler extracts canvas image and triggers a download
  const handleDownload = (chartRef) => {
    const canvas = chartRef.current.canvas;
    const url = canvas.toDataURL("image/png");
    const link = document.createElement("a");
    link.download = `occupancy-${new Date().toISOString().split("T")[0]}.png`;
    link.href = url;
    link.click();
  };

  return <Bar data={chartData} />;
};
export default OccupancyGraph;
```

Fig 5.2.3 – Occupancy Visualization

5.3 Hardware

ESP32 Core Hardware Functions

This code segment defines the core hardware functions that enable the IoT device to operate autonomously. The connectWiFi() function establishes a connection between the ESP32 and the local Wi-Fi network, retrying several times until a successful link is made. Once connected, the device can communicate with the cloud server. The checkFingerprint() function interacts with the R307 fingerprint sensor to capture, process, and identify a fingerprint. It compares the scanned fingerprint with stored templates to retrieve a corresponding user ID. When a valid match is found, the system identifies the authenticated faculty member. After successful authentication, the device sends a POST request containing the roomNumber, status, and fingerprintID to the backend endpoint /api/rooms/update, updating the room's occupancy status in real time. Additionally, two physical buttons allow manual toggling between "Vacant" and "Occupied" states, ensuring flexibility even without biometric input. The integration of Wi-Fi and fingerprint modules ensures both secure access and seamless cloud connectivity. This modular structure enhances system reliability and simplifies debugging or upgrades. Overall, it forms the foundation of the edge-level communication in the IoT-based occupancy monitoring system.

```ino
// hardware.ino (key functions)

// WiFi connect
void connectWiFi() {
  WiFi.begin(ssid, password);
  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts < 20) {
    delay(500); attempts++;
  }
  if (WiFi.status() == WL_CONNECTED) {
    // connected
  } else {
    // handle failure
  }
}

// Check fingerprint sensor and map to a user ID
int checkFingerprint() {
  uint8_t result = finger.getImage();
  if (result == FINGERPRINT_NOFINGER) return -2;
  if (result != FINGERPRINT_OK) return -2;

  result = finger.image2Tz();
  if (result != FINGERPRINT_OK) return -1;

  result = finger.fingerFastSearch();
  if (result != FINGERPRINT_OK) return -1;

  // finger.fingerID holds the matched sensor ID
  return finger.fingerID;
}
```

```ino
// Send room status update to backend
void updateRoomStatus(String status) {
  if (WiFi.status() != WL_CONNECTED) { resetAuthentication(); return; }

  HTTPClient http;
  http.begin(serverUrl);
  http.addHeader("Content-Type", "application/json");

  StaticJsonDocument<200> doc;
  doc["roomNumber"] = ROOM_NUMBER;
  doc["status"] = status;                    // "Vacant" or "Occupied"
  doc["fingerprintID"] = authenticatedUserID; // mapped user ID

  String payload;
  serializeJson(doc, payload);

  int httpCode = http.POST(payload);
  if (httpCode > 0) {
    String response = http.getString();
    // parse and display server message
  } else {
    // handle HTTP error
  }
  http.end();
  resetAuthentication();
}
```

Fig 5.3.1 – ESP32 Core Hardware Functions

Chapter 6

SYSTEM TESTING

6.1 Types of Testing

In the development of the Veriloc Classroom occupancy monitoring System, several types of testing were conducted to ensure system reliability, security, and performance. Each testing phase played a crucial role in validating different aspects of the system, ensuring it met both functional and non-functional requirements effectively.

Unit Testing: Individual modules and functions were tested in isolation. This included backend API endpoints, authentication middleware, database models, and frontend React components. This helped identify and fix minor logic and syntax issues early in the development process.

Integration Testing: The interaction between different modules was validated. This covered the end-to-end flow from IoT device data submission to database update and dashboard visualization. It ensured smooth communication between hardware, server, and client layers without data loss or inconsistency.

System Testing: The complete system was tested as a whole to verify that all requirements were met. This included real-time updates, multi-user support, and overall system stability. The tests confirmed that the system functioned correctly under realistic usage scenarios.

Performance Testing: The system was evaluated under load to measure response times, throughput, and scalability. Tests included concurrent user access and rapid IoT updates. Results showed that the system maintained optimal performance even under high activity levels.

Security Testing: Authentication mechanisms, data encryption, and access controls were tested to ensure protection against unauthorized access and common vulnerabilities. All security checks confirmed that sensitive data remained secure throughout communication and storage.

User Acceptance Testing (UAT): The system was deployed in a real-world classroom environment. Feedback was collected from actual users to validate usability and effectiveness. The users found the interface intuitive, and the overall system performance met their operational needs.

6.2 Test Cases and Screenshots

Test Case ID	Description	Input	Expected Output	Actual Output	Status
TC-01	Admin Login (valid)	Valid credentials	Lands to Landing Page	Lands to Landing Page	Pass
TC-02	Admin Login (invalid)	Invalid credentials	Back to Login Page	Back to Login Page	Pass
TC-03	Fingerprint Scan (valid)	Registered fingerprint	Access granted	Access granted	Pass
TC-04	Fingerprint Scan (invalid)	Unregistered fingerprint	Access denied Unknown finger	Access denied Unknown finger	Pass
TC-05	Room Status Update	POST from IoT device	Room status updated successfully	Room Status updated successfully	Pass
TC-06	Room Creation (valid)	Valid room details	Room created	Room created	Pass
TC-07	Room Creation (duplicate)	Duplicate time slot	Error message	Error shown	Pass
TC-08	Dashboard Update	Room status change	UI reflects new status	Real-time update	Pass
TC-09	Data Export	Click export button	CSV/PNG file generated	File downloaded	Pass
TC-10	API Security	Invalid/expired token	Access denied	Access denied	Pass
TC-11	Performance (load)	50 concurrent users	Stable operation	No failures	Pass
TC-12	Mobile Responsiveness	Access on mobile	Responsive layout	Layout adapts	Pass

Fig 6.2.1 –System Test Cases Table

Unit testing verifies individual components in isolation. For the Admin Login functionality (TC-01), we tested the login function independently to ensure proper JWT token generation and password

validation logic. The invalid login scenario (TC-02) was tested to verify error handling mechanisms, password comparison, and appropriate error message generation. In the Fingerprint Scan test (TC-03), we validated the fingerprint matching algorithm, template comparison process, and accurate user ID mapping. The Room Creation functionality (TC-06) underwent testing to verify room object creation, data validation rules, and proper database insertion logic.

Integration testing focuses on verifying interactions between different system components. The Room Status Update test (TC-05) examined the communication flow from IoT device to server, ensuring proper database updates and real-time event propagation. The Dashboard Update testing (TC-08) verified the seamless integration between frontend and backend systems, including WebSocket connections and state management updates across the application.

System testing ensures all components work together cohesively. The Room Creation duplicate test (TC-07) validated the complete room booking flow, including conflict detection and error handling across all system layers. Data Export functionality (TC-09) was tested end-to-end, ensuring proper file generation, download capabilities, and accurate data formatting across the entire system.

Security testing validates system protection against vulnerabilities. The invalid Admin Login test (TC-02) verified authentication barriers, credential validation, and protection against brute force attacks. For Fingerprint Scan security (TC-04), we tested biometric security measures and unauthorized access prevention at the hardware level. API Security testing (TC-10) focused on token validation, route protection, and proper implementation of authorization rules throughout the system.

Performance testing evaluates system behaviour under various load conditions. The load testing scenario (TC-11) validated the system's ability to handle multiple concurrent users while maintaining stability. This included comprehensive testing of resource utilization and response times under load, ensuring the system meets performance requirements even during peak usage periods.

User Acceptance Testing verifies that the system meets end-user requirements. The Mobile Responsiveness testing (TC-12) validated the system's adaptability across different devices and screen sizes. This included thorough testing of user workflow completion and interface usability, ensuring experience across all supported platforms and devices. Each testing phase was crucial in ensuring Veriloc’s reliability and user-friendliness. Comprehensive testing across multiple levels identified and resolved issues before deployment, resulting in a robust system.

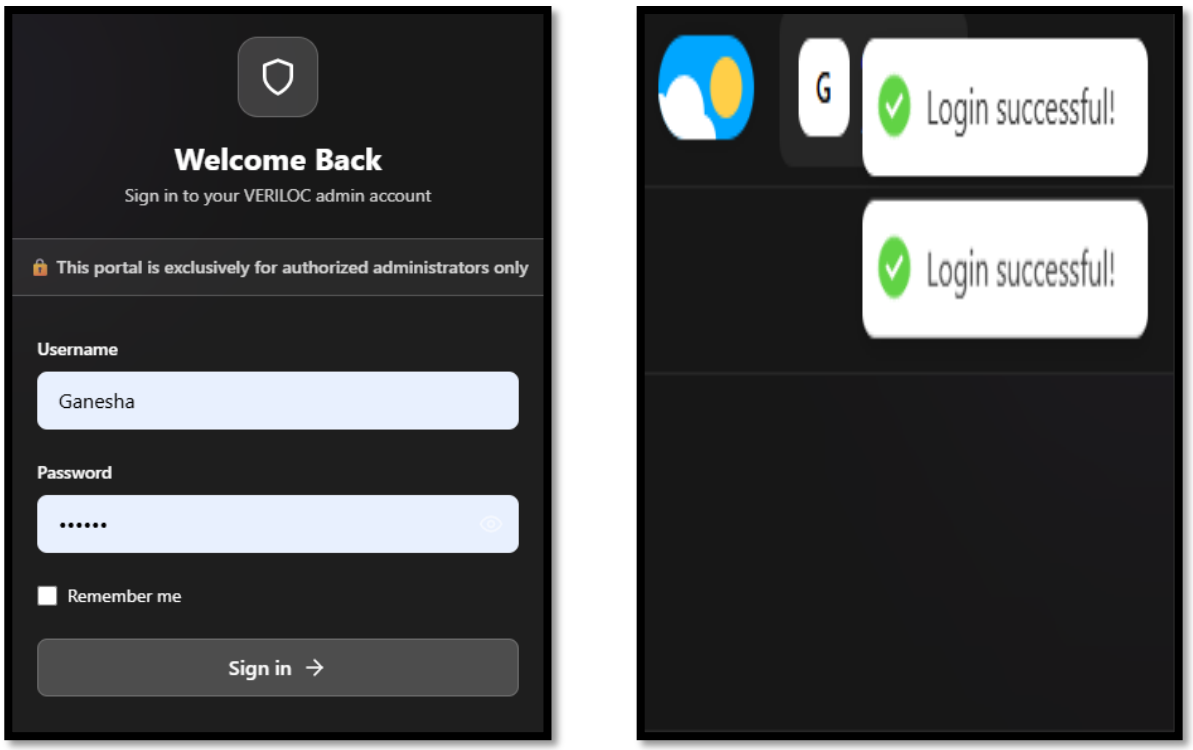


Fig 6.2.2 – Admin Login (Valid)

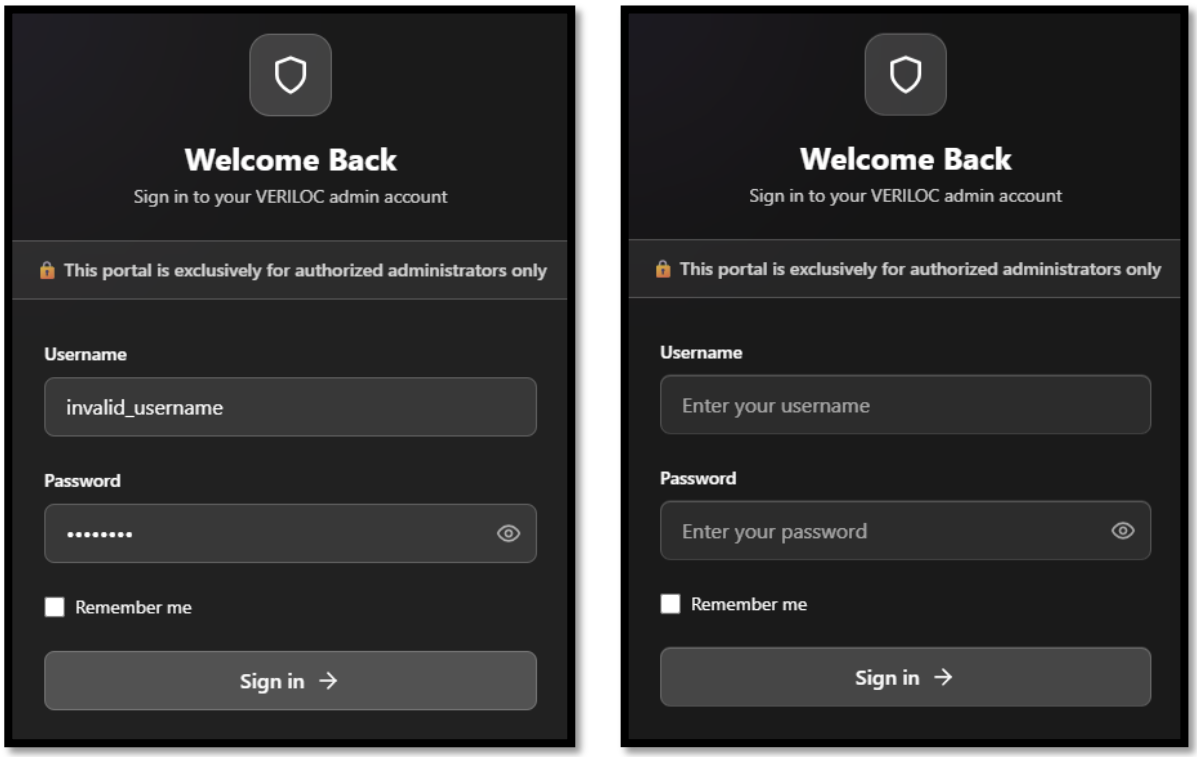


Fig 6.2.3 – Admin Login (Invalid)

Fig 6.2.4 – Fingerprint Scan (Invalid)

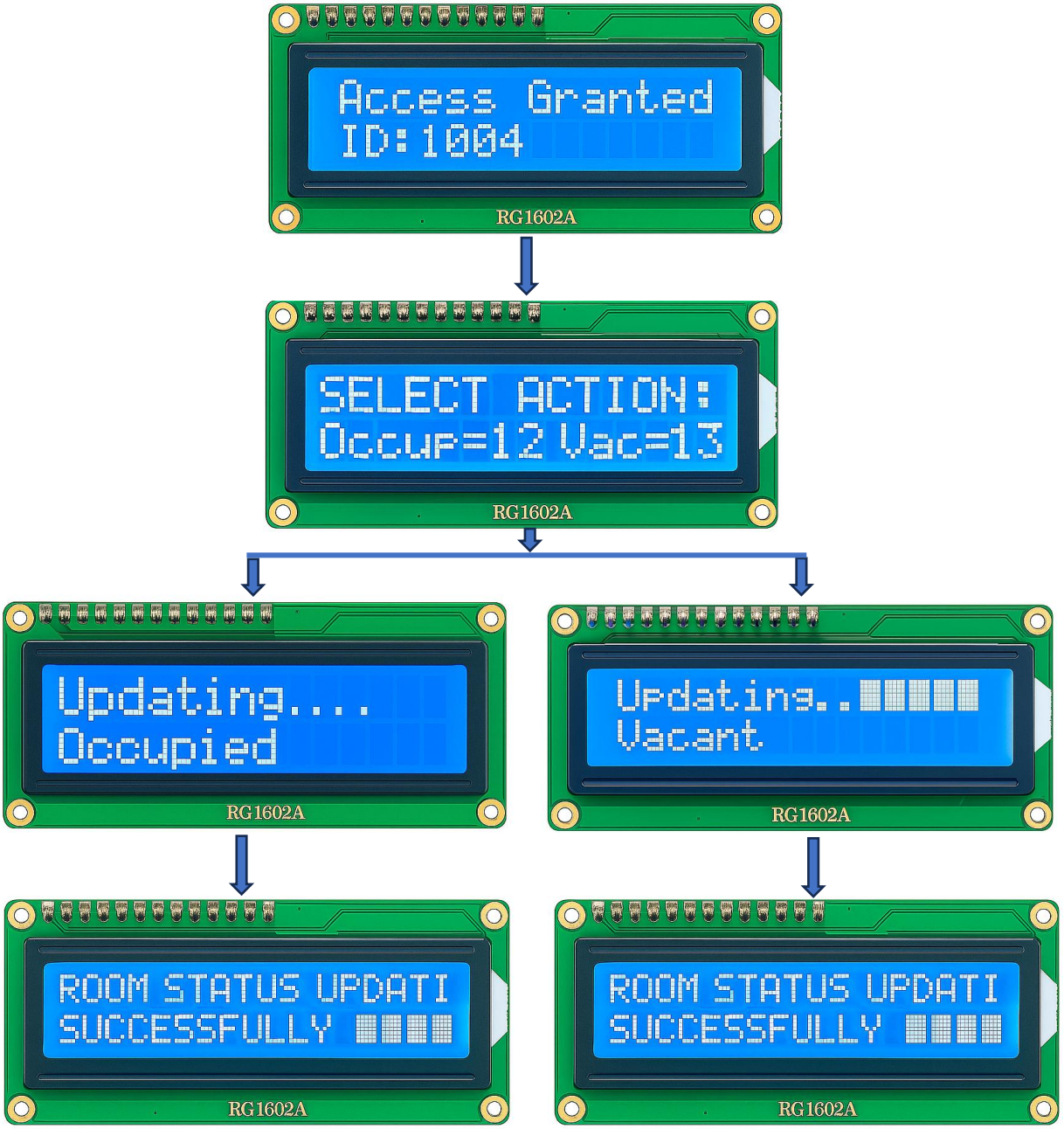


Fig 6.2.5 – Room Status Toggling Flow after successful scan

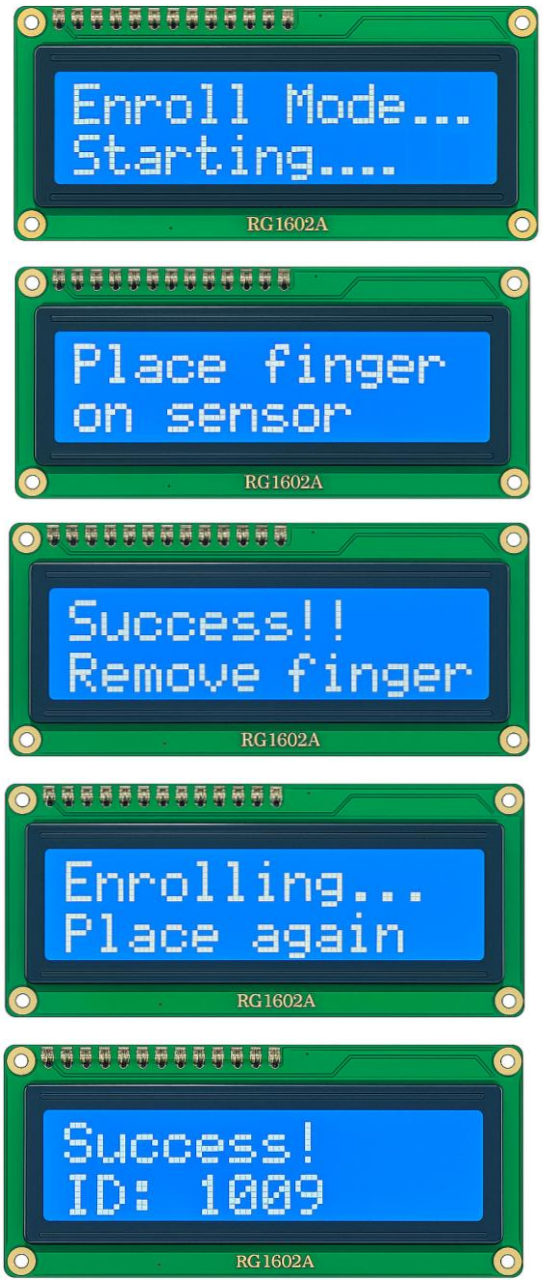
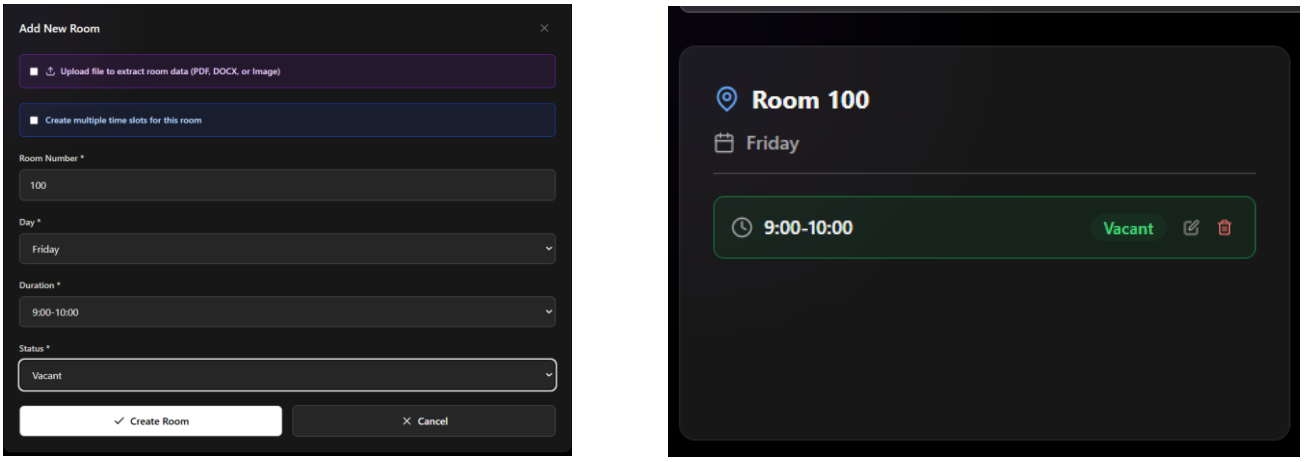


Fig 6.2.6 – Fingerprint Enrollment Process



\

Fig 6.2.7 – Adding of Classrooms

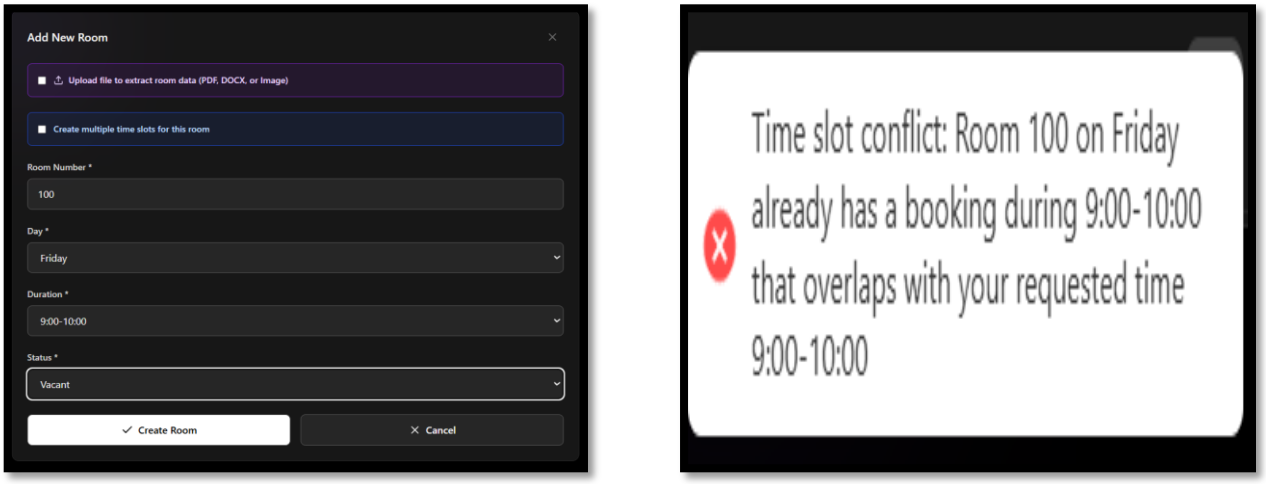


Fig 6.2.8 – Adding of Duplicate Classrooms at same duration

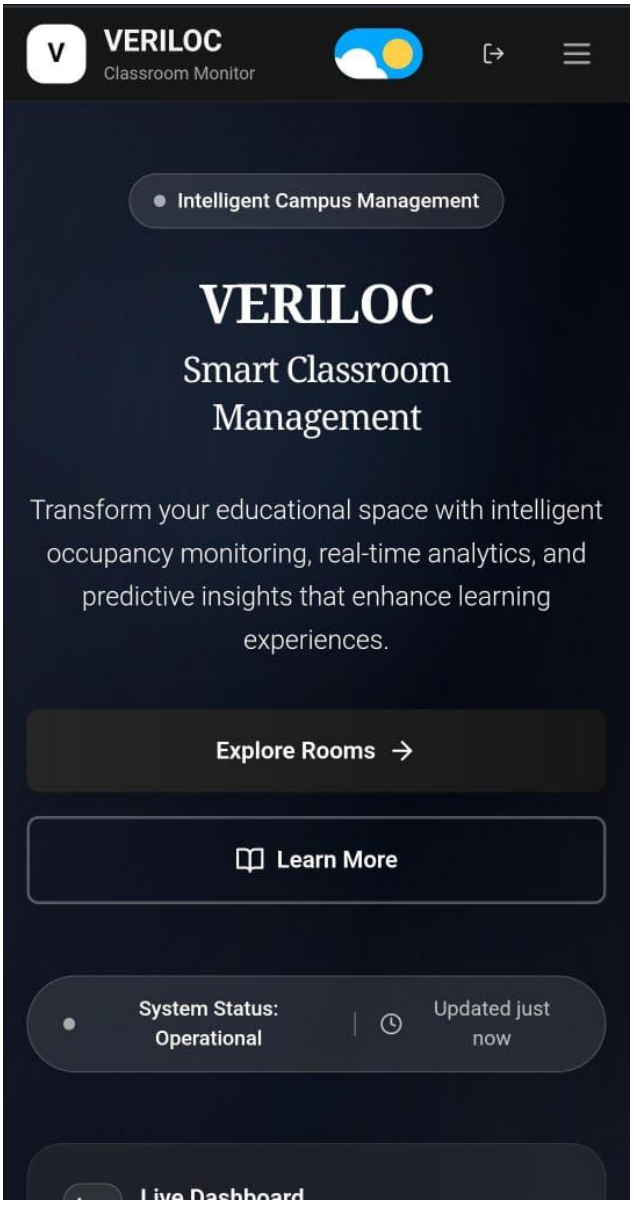


Fig 6.2.9 – Mobile Responsiveness

Chapter 7

RESULTS AND SCREENSHOTS

7.1 System Implementation Results

7.1.1 Hardware Implementation

The hardware components of the VERILOC system were successfully integrated and tested. The system comprises an ESP32 microcontroller, R307 fingerprint sensor, 16x2 LCD display with I2C interface, and tactile push buttons for manual control.

- **ESP32 Development Board:** The ESP32 module serves as the central processing unit, featuring built-in Wi-Fi connectivity for communication with the cloud server. The board includes multiple GPIO pins used for interfacing with sensors and displays, along with USB programming capability.
- **Tactile Push Buttons:** Two push buttons are used for manual status updates - one for "Vacant" and another for "Occupied." These provide a backup mechanism for status changes when fingerprint authentication is not required or unavailable.
- **16x2 LCD Display (Front View):** The LCD display shows real-time room status information to users. The blue backlight provides clear visibility, and the I2C interface simplifies wiring by requiring only four connections (VCC, GND, SDA, SCL).
- **LCD Display with I2C Module (Back View):** The back of the LCD shows the I2C adapter module (PCF8574), which converts parallel LCD signals to I2C protocol. This module includes a potentiometer for contrast adjustment and reduces the required GPIO pins from 6 to just 2 (SDA and SCL).
- **Integrated Hardware Prototype:** This is a compact biometric-access / attendance-style hardware module combining a fingerprint reader, 16x2 character LCD, two push buttons, and

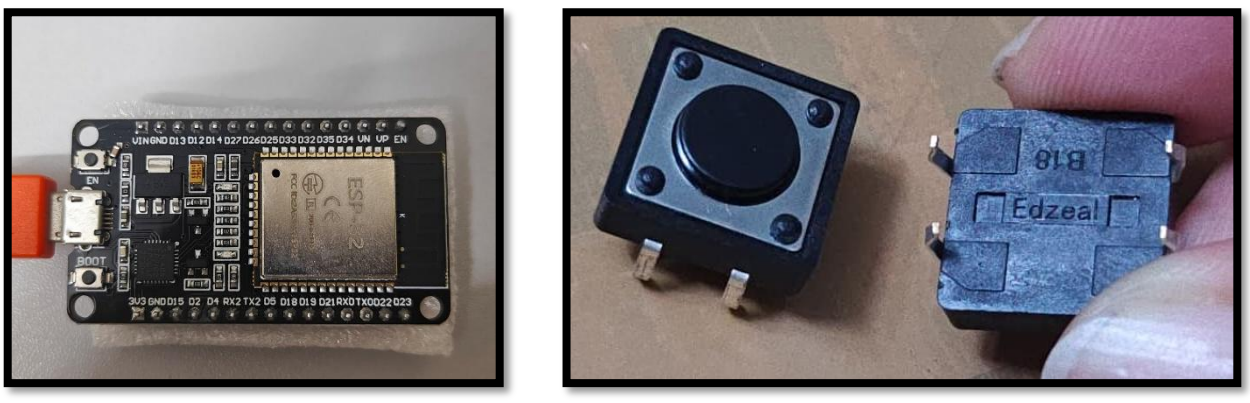
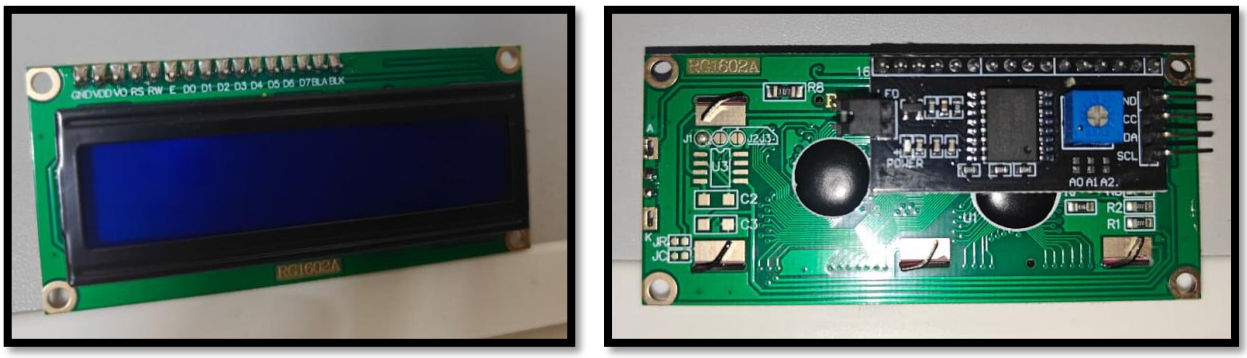


Fig 7.1.1.1 ESP32 MC and Tactile Push Buttons



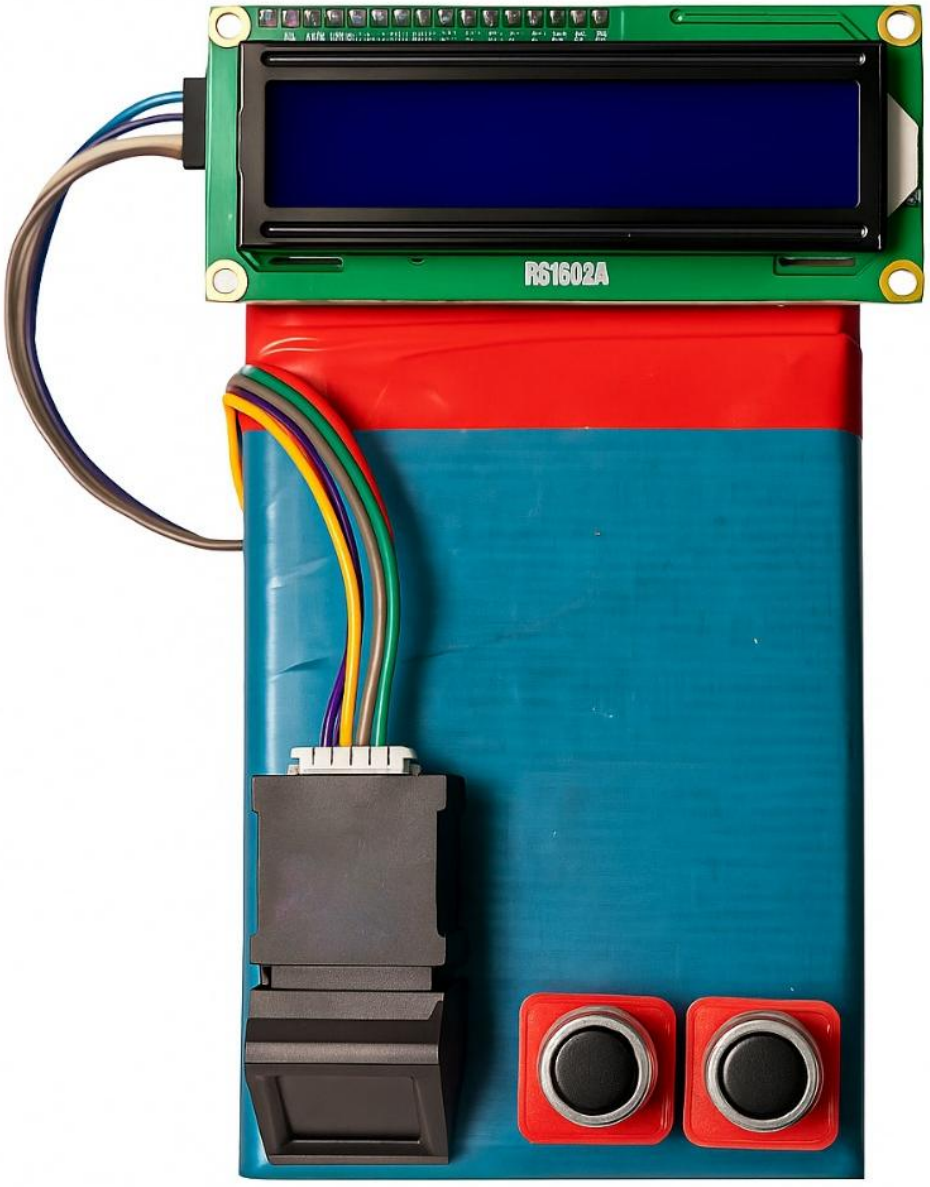


Fig 7.1.1.2 LCD and Integrated Hardware Prototype

a rechargeable battery pack. It looks like an embedded user-interface unit for enrolment/verification and local feedback (display + buttons), powered by a Li-ion battery and controlled by a microcontroller (hidden on the back or inside). Typical use - enroll users' fingerprints, verify presence, and display status/messages on the LCD (e.g., "Place finger", "Match OK", user id/time). Buttons provide manual actions (enrol, cancel, menu). The fingerprint module performs capture + matching and returns an ID or match result to the MCU.

Inputs: Fingerprint scan (biometric template), Button presses, Power (battery / charger).

Outputs: LCD text messages (status, prompts), Stored event/attendance records.

Success criteria: The system should reliably enroll and verify fingerprints with minimal errors, provide clear real-time feedback through LCD/LED/buzzer, and operate safely on battery without data loss during charging.

7.1.2 Web Application Interface

The VERILOC web application provides a comprehensive dashboard for monitoring and managing classroom occupancy across the campus.

Home Dashboard (Room Availability): The main dashboard displays real-time room status with advanced filtering options. Users can search by room number, filter by day of the week, duration, and occupancy status. The interface shows 63 total rooms with color-coded status indicators (green for vacant, red for occupied). The live analytics panel on the right provides a weekly occupancy trends graph, helping administrators visualize usage patterns. The "Live" indicator confirms real-time data synchronization with IoT devices.

Admin Login Page: The secure login interface restricts access to authorized administrators only. The authentication system uses JWT tokens for session management and includes username/password validation. The "Remember me" feature enhances user experience for frequent logins.

Landing Page: The landing page presents Veriloc as an intelligent campus management solution with the tagline "Smart Classroom Management." It displays real-time statistics showing 63 total rooms, 62 available, and 1 occupied. Key features highlighted include Secure Access Control, Real-time Updates, live monitoring, Analytics and reports and system active status.

Admin Management Page: This interface allows super admins to add new faculty members to the system. The form captures username, email, password, and fingerprint ID. Step-by-step instructions guide users through the fingerprint enrolment process: (1) Enrol fingerprint on R307 sensor, (2) Note

the 4-digit ID displayed on OLED, (3) Enter the ID in the Fingerprint ID field. This ensures proper synchronization between software and hardware systems.

Room Management Page: Administrators can create new rooms by specifying room number, day of operation, duration slot, and initial status. The interface displays existing rooms in a card layout showing room number, scheduled day, and time slots with status. Edit and delete options enable easy room data management. The search and filter functionality helps quickly locate specific rooms from the complete inventory.

Faculty Attendance Tracker: This specialized module tracks faculty presence using fingerprint authentication data. For the selected date (31-10-2025), the dashboard shows total faculty count, present/absent statistics, and attendance percentage. The detailed table lists each faculty member with their email, fingerprint ID, current status, number of activities, and first activity timestamp. The "Export PDF" feature enables easy report generation for administrative records.

Analytics Dashboard: The advanced analytics page provides comprehensive insights into classroom utilization. The bar chart displays weekly occupancy trends with separate visualization for vacant (green) and occupied (red) rooms across all weekdays. The "Chart Controls" and "Download" options allow data manipulation and export for further analysis. This data-driven approach helps administrators optimize classroom allocation and identify underutilized spaces

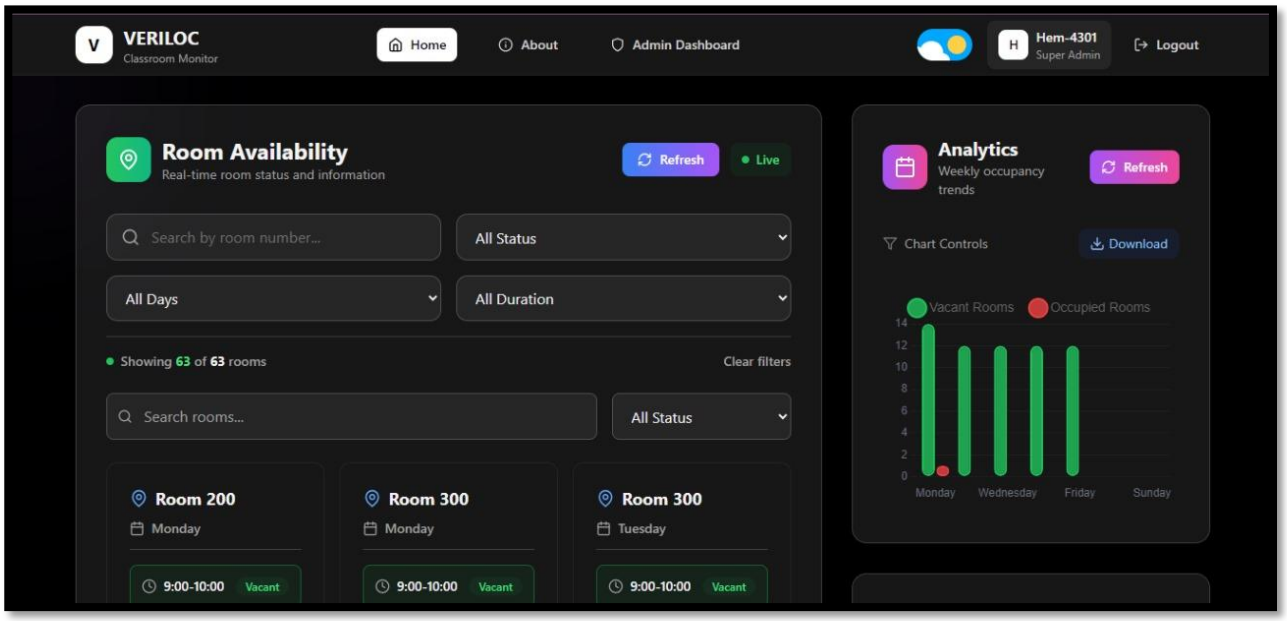


Fig 7.1.2 – Home Dashboard

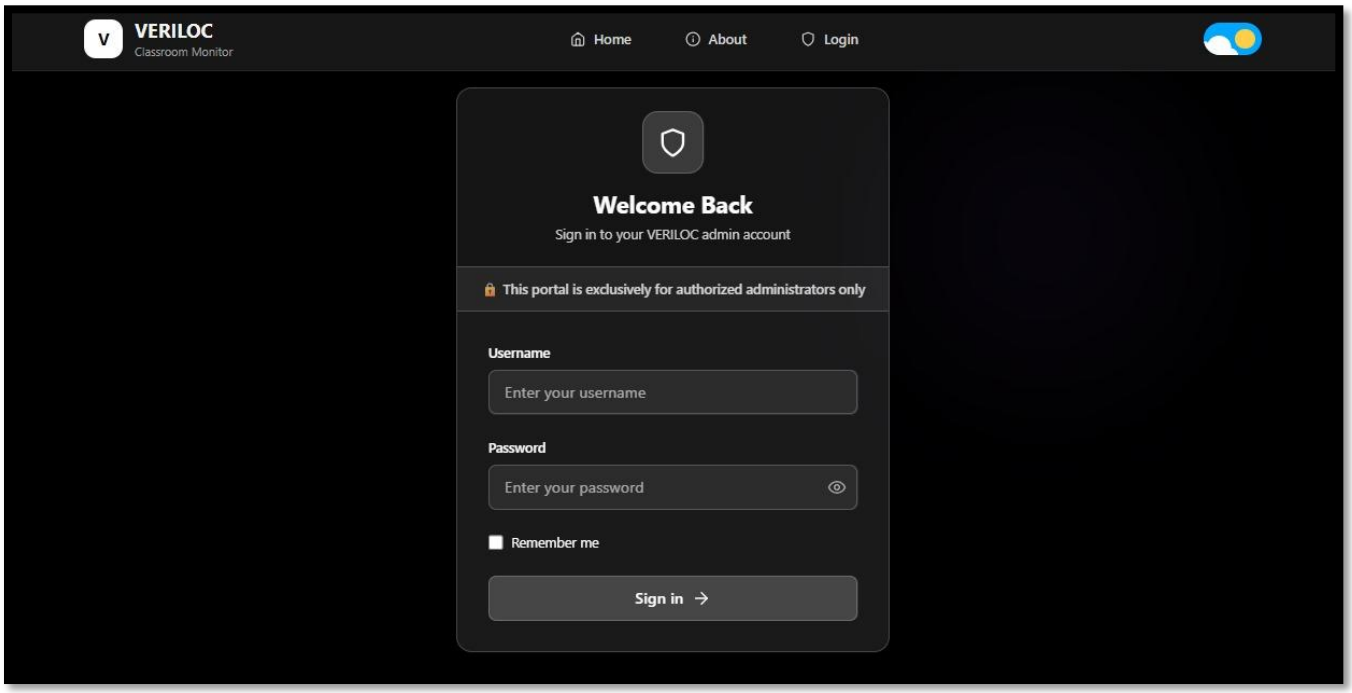


Fig 7.1.2.2 – Login Page and Landing Page

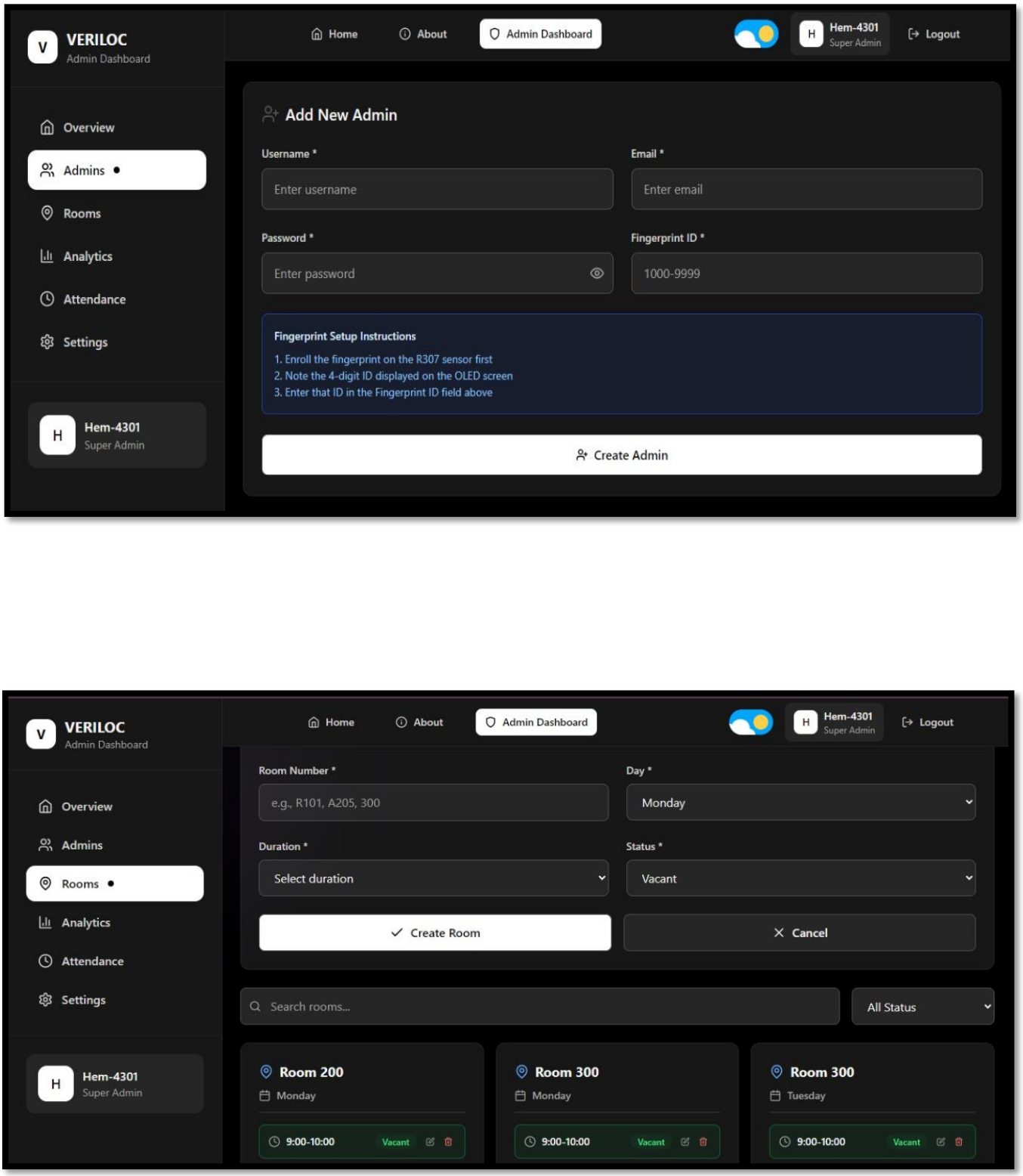
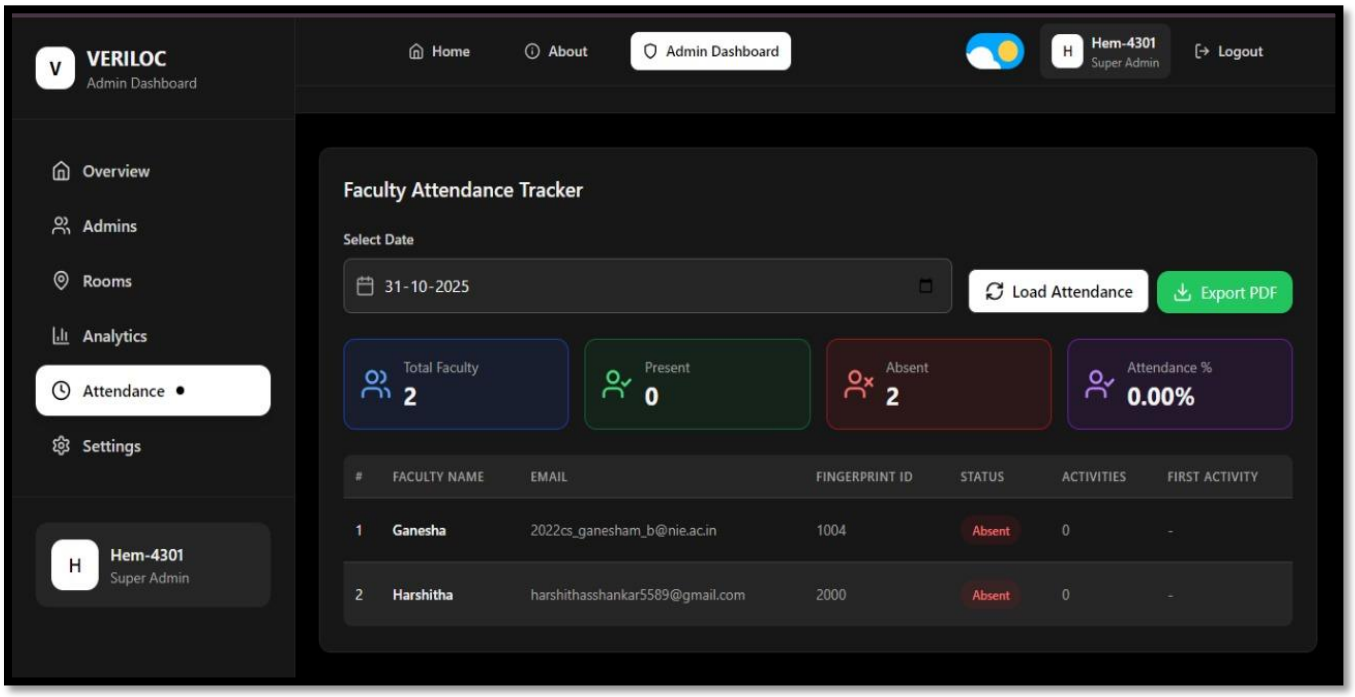


Fig 7.1.2.3 – Admin Management Page and Room Management Page



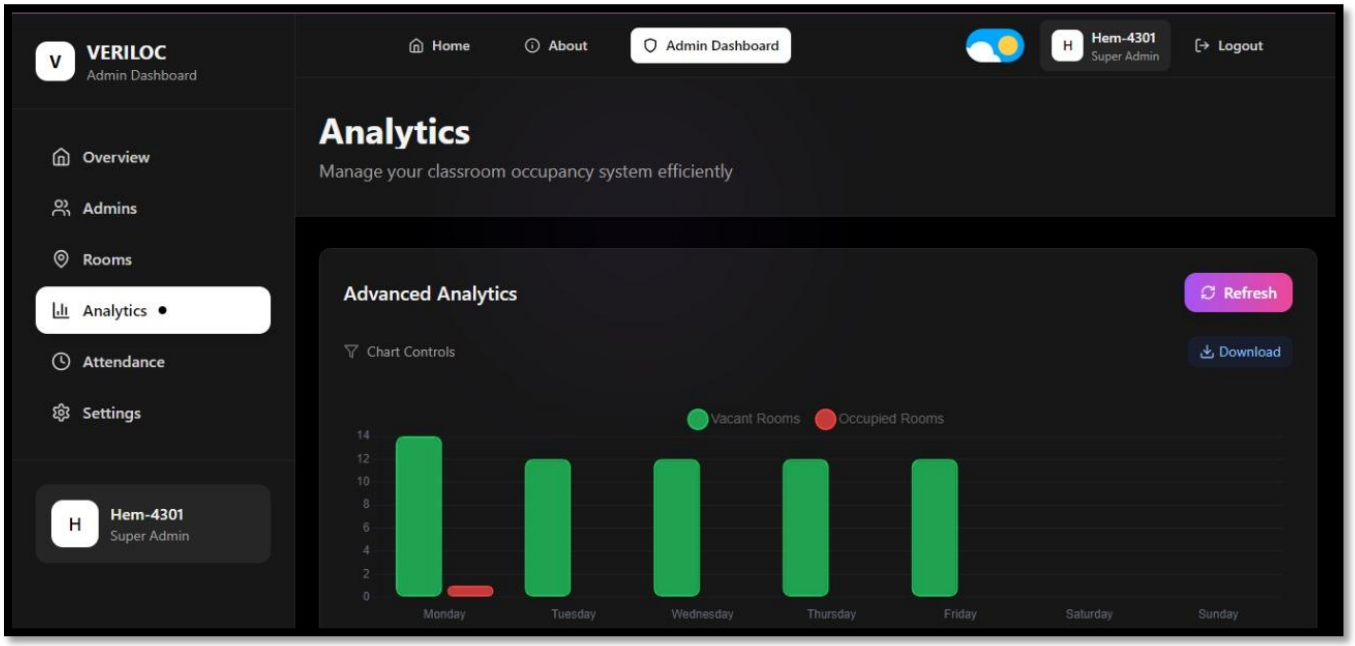


Fig 7.1.2.4 – Faculty Attendance Tracker and Analytics Dashboard

Chapter 8

CONCLUSION AND FUTURE ENHANCEMENTS

Conclusion

The VERILOC (Verified Location) Classroom Occupancy Tracker successfully implements a comprehensive solution for real-time classroom occupancy monitoring using IoT integration and biometric verification. Built on the MERN (MongoDB, Express.js, React.js, Node.js) stack with ESP32 microcontroller integration, the system provides a secure and efficient platform for classroom status management. Key achievements include:

- **Biometric Security:** Implementation of fingerprint authentication ensures only authorized personnel can update room status, maintaining data integrity and security.
- **Real-time Monitoring:** The system provides instant status updates through IoT integration, enabling immediate visibility of classroom occupancy across the institution.
- **User-friendly Interface:** A responsive web dashboard offers intuitive room management, detailed analytics, and comprehensive activity logging for administrators.
- **Robust Architecture:** The combination of hardware (ESP32 with R307 fingerprint sensor) and software (MERN stack) create a reliable and scalable system for classroom management.

Future Enhancements

Several potential improvements could further enhance VERILOC's functionality and user experience. Future updates aim to make the system smarter, more efficient, and easier to use.

- **Technical Enhancements:** VERILOC can be enhanced technically by integrating machine learning for predicting occupancy trends and generating automated reports. Native mobile apps for iOS and Android can enable push notifications and offline access. Expanding hardware with sensors like temperature, motion, and smart door locks will further improve automation.
- **Functional Improvements:** Future updates can include automated email/SMS alerts, in-app communication, and integration with facility systems. upgrades such as multi-tenant support, customizable reports, and advanced booking will make the system more flexible and efficient.
- **User Experience Enhancements:** The dashboard can be improved with customizable widgets, better search features, and interactive floor plans. Accessibility features like multilingual support, voice commands, and screen reader compatibility will enhance usability for all users.

Chapter 9

REFERENCES

1. **"Occupancy Prediction in IoT-Enabled Smart Buildings: Technologies, Methods, and Future Directions,"** *Sensors*, vol. 24, no. 11, p. 3276, **2024**.
2. **"Testing and Evaluation of Low-Cost Sensors for Developing Open Smart Campus Systems Based on IoT,"** *Sensors*, vol. 23, no. 20, p. 8652, **2023**.
3. **"Is IoT Monitoring Key to Improve Building Energy Efficiency? Case Study of a Smart Campus in Spain,"** *Energy and Buildings*, vol. 285, p. 112882, Apr. **2023**.
4. **"Toward an Intelligent Campus: IoT Platform for Remote Monitoring and Control of Smart Buildings,"** *Sensors*, vol. 22, no. 23, **2022**.
5. **R. Singh and M. Kaur,** "Smart Classroom Occupancy Detection Using IoT Sensors," *IEEE Access*, **2021**.
6. **W. Abassi, I. Ahmad, and M. B. Iqbal,** "Energy Conservation Smart Classroom System using IoT," *International Journal of Advancements in Mathematics*, vol. 1, no. 1, pp. 32–46, Dec. **2021**.
7. **S. Sharma and A. Patel,** "Real-Time Attendance and Availability Systems in Smart Environments," in *Proceedings of IEEE International Conference on Smart Environments*, **2020**.
8. **L. Chen, X. Zhao, and Y. Wang,** "An IoT Framework for Real-Time Resource Scheduling in Educational Institutions," *Journal of Cloud Computing*, **2020**.
9. **P. Paudel, S. Kim, S. Park, and K.-H. Choi,** "A Context-Aware IoT and Deep-Learning-Based Smart Classroom for Controlling Demand and Supply of Power Load," *Electronics*, vol. 9, no. 6, p. 1039, **2020**.
10. **H. Al-Rimy and M. F. Zolkipli,** "Secure IoT Authentication Using Biometric Sensors," *IEEE Transactions on Information Forensics and Security*, **2020**.
11. **P. Kumar,** "Design and Implementation of a Biometric Authentication System for Classroom Access," *International Journal of Computer Applications*, **2019**.
12. **S. Das and S. Banerjee,** "Low-Cost Embedded Systems for Smart Campus Infrastructure," *Springer IoT Journal*, **2018**.